UET.CS2043 - Lập trình nâng cao

Thêm về Java

- Một số kiến thức về Java

Chương trình Java

- Chương trình Java bao gồm một tập các đối tượng
- Mỗi lớp được đặc tả trong một tệp nguồn (tên tệp trùng với tên lớp)
  - Mỗi dòng lệnh trong Java đều thuộc một lớp (không tính các lệnh import)
  - Tăng tính mô-đun hóa
  - Dễ hơn cho sửa đổi mã nguồn, giảm thời gian biên dịch

khai báo Lớp
bắt đầu/kết thúc

02 lớp
trong 02 tệp

lớp được khai báo **public** phải được lưu trong tệp cùng tên lớp

```java
// GradeBook.java
public class GradeBook
{
   // display a welcome message to the GradeBook user
  public void displayMessage()
  {
    System.out.println( "Welcome to the Grade Book!" );
  }
} // end class GradeBook
```

main() được gọi tự động bởi máy ảo Java khi chương trình được thực thi

```java
// GradeBookTest.java
public class GradeBookTest
{
  // main method begins program execution
  public static void main( String args[] )
  {
    // create a GradeBook object and assign it to myGradeBook
    GradeBook myGradeBook = new GradeBook();
    // call myGradeBook's displayMessage method
    myGradeBook.displayMessage();
  }
} // end class GradeBookTest
```

tạo một thể hiện (đối tượng) của lớp và được tham chiếu bởi myGradeBook

```
Welcome to the Grade Book!
```

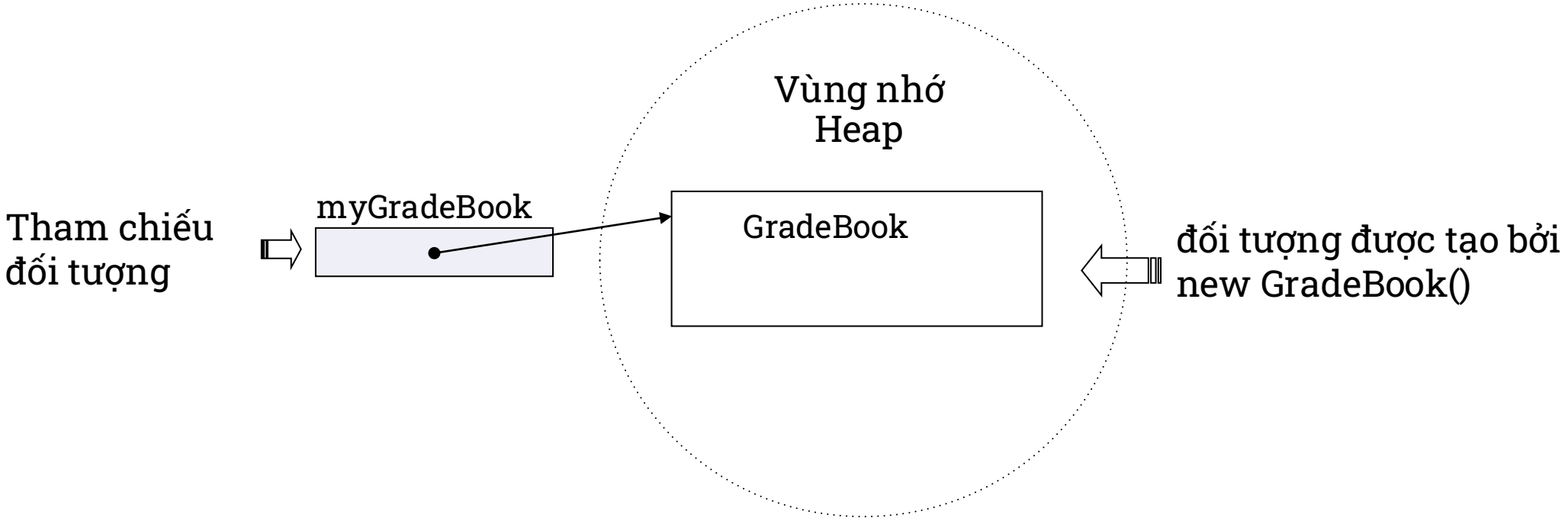
Các đối tượng

- Các đối tượng được thao tác qua các tham chiếu (references)
  - các tham chiếu đối tượng đóng vai trò giống như các con trỏ
- Các đối tượng phải được tạo tường minh bởi toán tử new

```
public class GradeBookTest {
  // phương thức main bắt đầu thực thi chương trình
  public static void main( String args[] ) {
    // tạo đối tượng GradeBook và gán cho myGradeBook
    GradeBook myGradeBook = new GradeBook();
    // gọi phương thức displayMessage của myGradeBook
    myGradeBook.displayMessage();
  }
} // kết thúc lớp GradeBookTest
```

Đối tượng và Tham chiếu đối tượng

```
...
    // tạo đối tượng GradeBook và gán cho myGradeBook
    GradeBook myGradeBook = new GradeBook();
...
```



Thuộc tính, Phương thức và Kiểm soát truy cập

- Các phạm vi truy cập
  - Public
    - Có thể truy cập ở bất kỳ đâu và bởi bất kỳ ai
  - Protected
    - Chỉ có thể truy cập bởi lớp đó và các lớp con hoặc các lớp trong cùng gói (package)
  - Private
    - Chỉ có thể truy cập bởi lớp đó

GradeBook
- courseName : String
+ setCourseName(name : String) + getCourseName() : String + displayMessage()

private:
chỉ được truy cập bởi các phương thức của lớp

```java
// GradeBook.java
public class GradeBook
{
  private String courseName; // course name for this GradeBook

  // method to set the course name
  public void setCourseName( String name )
  {
    courseName = name; // store the course name
  }
  // method to retrieve the course name
  public String getCourseName()
  {
    return courseName;
  }
  // display a welcome message to the GradeBook user
  public void displayMessage()
  {
    System.out.println( "Welcome to the Grade Book!" );
  }
} // end class GradeBook
```

thuộc tính (attribute):
Mỗi đối tượng **Gradebook** có biến thể hiện tên **courseName**

các phạm vi truy cập

các phương thức

```java
class MyDate {
    private int year, mon, day;
    public int getYear() {
        return year;
    }
    public void setYear(int y){
  year=y;
    }
}
```

```java
MyDate d = new MyDate();
...
d.year = 2005;    // compile error

d.setYear(2005);
System.out.println(d.getYear());
```

Nạp chồng phương thức (overloading)

- Có thể định nghĩa các phương thức trùng tên nhưng phải khác danh sách tham số

```
class MyDate {
    …
    public boolean setMonth(int m) { …}
    public boolean setMonth(String s) { …}
}
…
d.setMonth(9);
d.setMonth(”September”);
```

Phương thức khởi tạo (constructor)

- Dữ liệu nên được khởi tạo trước khi sử dụng
  - lỗi khởi tạo là một trong các lỗi phổ biến
- Phương thức khởi tạo
  - là phương thức đặc biệt được gọi tự động ngay sau khi đối tượng được tạo ra
  - nhằm mục đích chính là khởi tạo cho các thuộc tính của đối tượng

Phương thức khởi tạo

- Tên trùng với tên lớp
- Không nhận giá trị trả lại
- Được gọi tự động mỗi khi đối tượng được tạo (bởi toán tử new)
  - nếu không khai báo, hệ thống sẽ gọi phương thức khởi tạo mặc định - phương thức rỗng

GradeBook
- courseName : String
«constructor» GradeBook(name: String) + setCourseName(name: String) + getCourseName() : String + displayMessage()

```java
// GradeBook.java
public class GradeBook
{
  private String courseName; // course name

  // constructor initializes courseName
  public GradeBook( String name )
  {
    courseName = name; // initializes courseName 12
  }
  // method to set the course name
  public void setCourseName( String name )
  {
    courseName = name; // store the course name
  }
  // method to retrieve the course name
  public String getCourseName()
  {
    return cour
  }
  // display a
  public void d
  {
    System.out.
  }
} // end class GradeBook
```

```java
// GradeBookTest.java
…
    // tạo đối tượng GradeBook và gán cho myGradeBook
    GradeBook myGradeBook =
       new GradeBook( "CS101 Introduction to Java Programming" );
…
```

Ví dụ: Phương thức khởi tạo rỗng

- Phương thức khởi tạo không được khai báo tường minh

```
class MyDate {
}
…

MyDate date = new MyDate();
```

Ví dụ: Phương thức khởi tạo mặc định

- Phương thức khởi tạo không nhận tham số

```
class MyDate {
    private int day, month, year;
    public MyDate() {
        day=1;
        month=1;
        year=2014;
        System.out.println("A new date created");
    }
}
...
MyDate d= new MyDate();
```

Ví dụ: Nhiều phương thức khởi tạo

```
class MyDate {
    private int day, month, year;
    public MyDate() {
        day=1;month=1;year=2014;
        System.out.println("A new date created");
    }
    public MyDate(int d,m,y) {
        day=d;
        month=m;
        year=y;
    }
}
...
MyDate d1 = new MyDate();
MyDate d2 = new MyDate(19,9,2014);
```

Ví dụ: Khi không có phương thức khởi tạo mặc định

```
class MyDate {
    private int day, month, year;
    public MyDate(int d,m,y) {
      day=d;
        month=m;
        year=y;
    }
}
...
MyDate d1 = new MyDate();      //error
MyDate d2 = new MyDate(19,9,2014);
```

Khởi tạo sao chép (copy constructor)

- Khởi tạo đối tượng bằng một đối tượng khác

```
public class MyDate {
    private int year, month, day;
    public MyDate() {...}
    public MyDate(MyDate d) {
        year = d.year;
        month = d.month;
        day = d.day;
    }
...
}
```

Ví dụ khởi tạo sao chép

```
MyDate d = new MyDate();
d.setYear(2005);
d.setMonth(9);
d.setDay(12);
MyDate openDay = new MyDate(d);
MyDate dd = d;
dd = new MyDate();
```

Các kiểu dữ liệu

- Java là ngôn ngữ định kiểu mạnh
  - Tất cả các biến đều phải có kiểu
- Kiểu dữ liệu nguyên thủy
  - Các biến được thao tác thông qua tên biến
    - int a = 5;
    - if (a == b)...
- Các kiểu tham chiếu (references)
  - Tham chiếu đến các đối tượng
  - Các đối tượng được thao tác qua các tham chiếu
    - GradeBook myGradeBook = new GradeBook();

Các kiểu dữ liệu nguyên thủy

- Các kiểu dữ liệu nguyên thủy trong Java
  - Numerical: byte, int, long, float, double
    - No unsigned
    - Same size in all platforms
  - Logical: boolean (true/false)
  - Characters: char
- Dữ liệu nguyên thủy KHÔNG PHẢI là các đối tượng
  - int count = 0;
  - if (count == 5) ...
- Có các lớp tương ứng gói kiểu nguyên thủy (wrapper classes)
  - Integer, Float, ...
  - Integer count = new Integer(0);

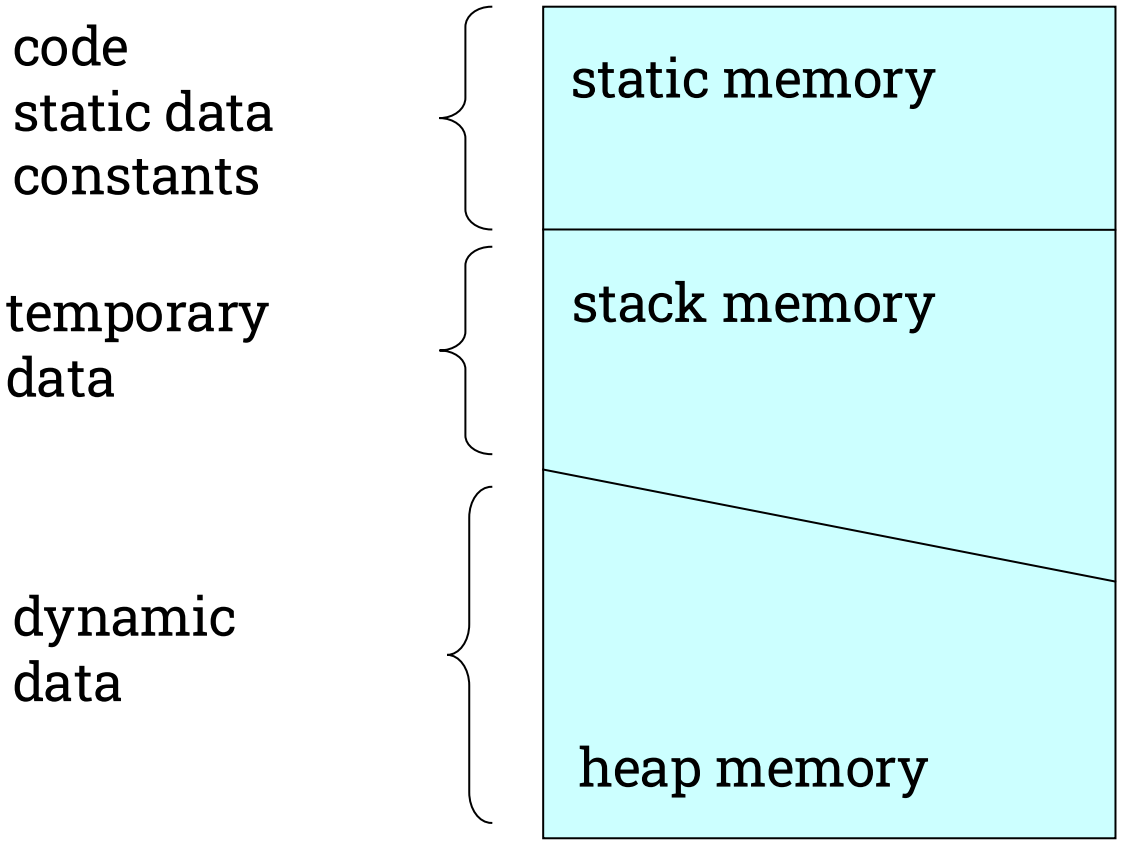
Các kiểu dữ liệu nguyên thủy

Kiểu	Kích cỡ	Giá trị cực tiểu	Giá trị cực đại	Lớp tương ứng
char	10 bits	0x0	0xffff	Character
byte	8 bits	-2^7	$2^7 - 1$	Byte
short	16 bits	-2^8	$2^8 - 1$	Short
int	32 bits	-2^{31}	$2^{31} - 1$	Integer
long	64 bits	-2^{63}	$2^{63} - 1$	Long
float	32 bits	1.40129846432481707e-45	3.40282346638528860e+38	Float
double	64 bits	4.94065645841246544e-324	1.79769313486231570e+308	Double
boolean	-			Boolean

Dữ liệu được lưu ở đâu?

- Dữ liệu nguyên thủy, các tham chiếu và các đối tượng được lưu ở đâu?
  - Registers: các lập trình viên không thể can thiệp
  - Stack: lưu các tham chiếu đối tượng
  - Heap: lưu các đối tượng
  - Vùng tĩnh (static storage): dữ liêu dùng trong suốt quá trình chương trình thực thi
  - Vùng hằng (constant storage): giá trị hằng thường được lưu trực tiếp trong phần mã nguồn chương trình
  - Vùng Non-RAM: lưu các luồng

Các vùng bộ nhớ cho ứng dụng



Bộ nhớ Stack và Heap

Các tham chiếu (references)

- Lưu “địa chỉ” đối tượng trong bộ nhớ máy tính
- Đối tượng được thao tác qua tham chiếu
  - Con trỏ đến đối tượng
  - Điều khiển trực tiếp các thuộc tính và phương thức
  - Không có các toán tử con trỏ
  - Phép gán (=) không sao chép nội dung đối tượng
- Được lưu trong vùng nhớ stack/static

Toán tử New

- Phải tạo mọi đối tượng một cách tường minh bằng toán tử new
  - cấp phát vùng nhớ động
  - được tạo trong bộ nhớ Heap
- Ví dụ:

```
MyDate d;
d = new MyDate();
```

Phép gán "="

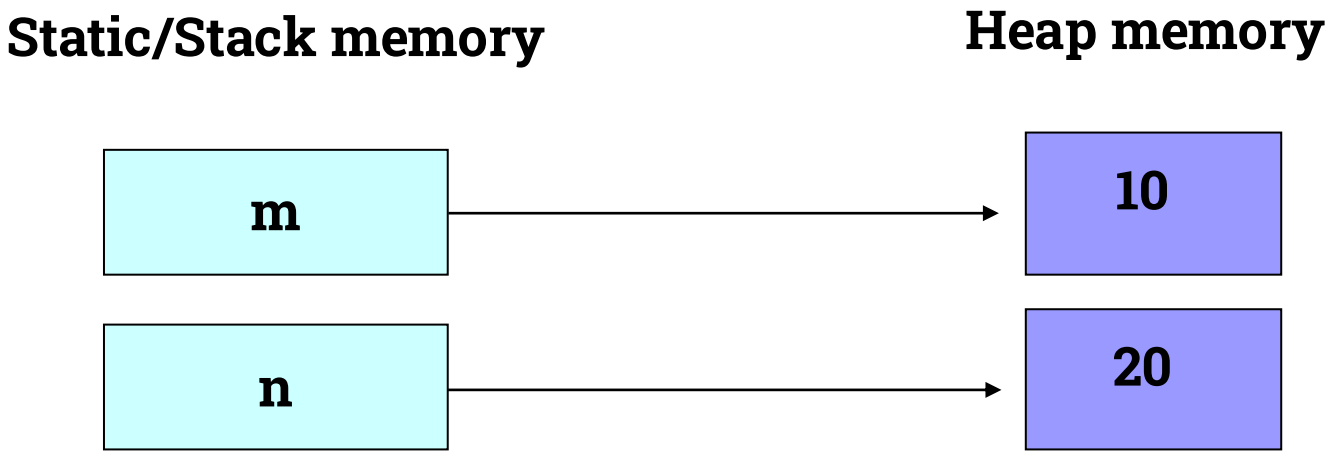
- Phép gán không sao chép như thông thường
  - sao chép nội dung của tham chiếu
  - 2 tham chiếu sẽ tham chiếu đến cùng đối tượng

```
Integer m  = new Integer(10);
Integer n  = new Integer(20);
m = n;
n.setValue(50);
System.out.print(m);
```

```
50
```

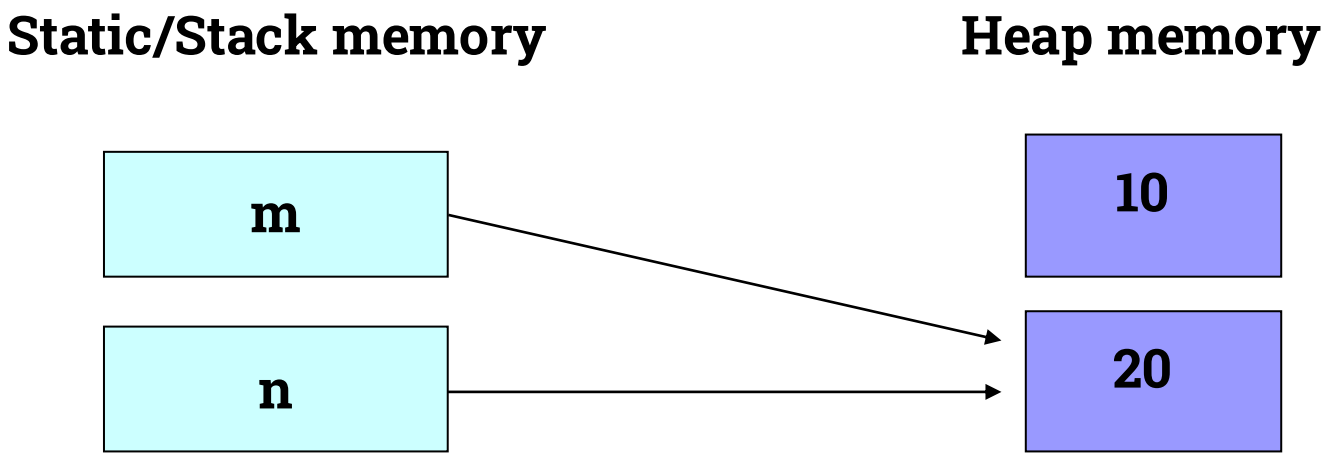
Ví dụ về “new” và “=”

```
Integer m  = new Integer(10);
Integer n  = new Integer(20);
m = n;
n.setValue(50);
```



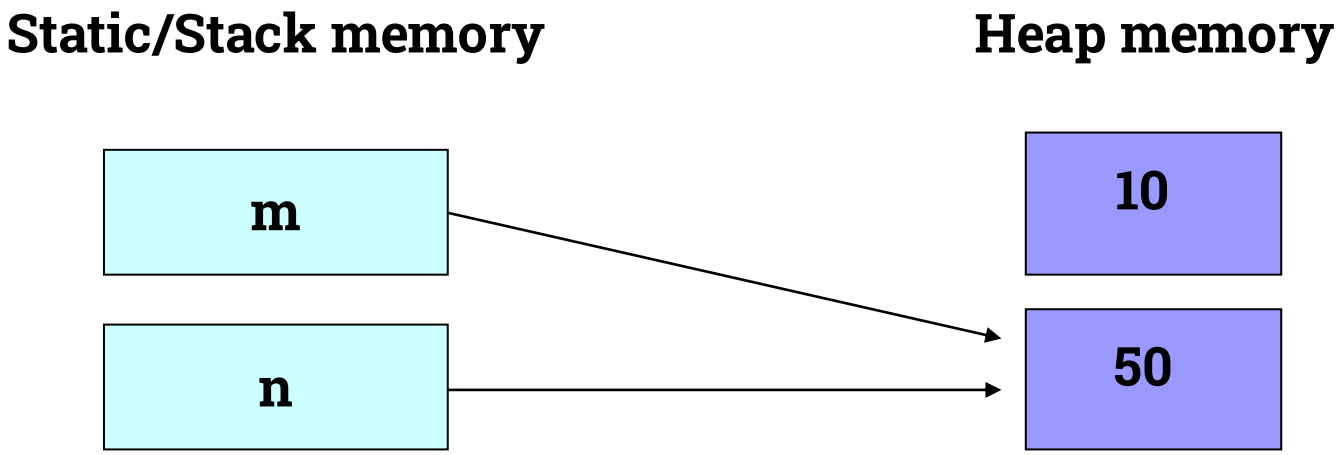
Ví dụ về “new” và “=”

```
Integer m  = new Integer(10);
Integer n  = new Integer(20);
m = n;
n.setValue(50);
```



Ví dụ về “new” và “=”

```
Integer m  = new Integer(10);
Integer n  = new Integer(20);
m = n;
n.setValue(50);
```



Toán tử "==" và "!="

- So sánh nội dung các biến
  - Giá trị của dữ liệu nguyên thủy
  - Giá trị của các tham chiếu
    - chỉ kiểm tra xem có tham chiếu đến cùng đối tượng
    - KHÔNG so sánh nội dung các đối tượng

```
Integer m1  = new Integer(10);
Integer m2  = new Integer(10);
System.out.println(m1 == m2);

int n1 = 1;
int n2 = 1;
System.out.println(n1 == n2);
```

```
false
true
```

So sánh nội dung đối tượng

- Phương thức "equals"
  - Được định nghĩa trước
    - Sẵn sàng để dùng
  - Do người dùng định nghĩa
    - equals() phải được định nghĩa, nếu không sẽ trả lại false
    - ghi đè / overriding (xem bài sau)

```
Integer m1 = new Integer(10);
Integer m2 = new Integer(10);
System.out.print(m1.equals(m2));
```

```
class MyInteger {
    private int value;
    public boolean equals (MyInteger other) {
        return (value == other.value);
    }
...
}
```

```
...
MyInteger m1 = new MyInteger(10);
MyInteger m2 = new MyInteger(10);
System.out.print(m1.equals(m2));
```

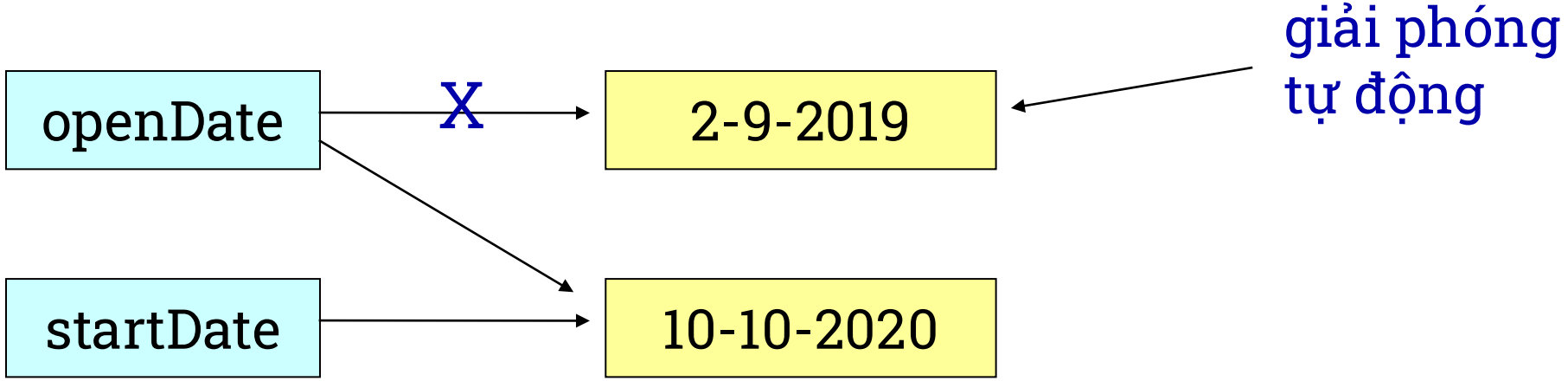
Thu hồi bộ nhớ (Garbage collection)

- Lập trình viên không cần phải giải phóng đối tượng
- JVM cài đặt cơ chế "Garbage collection" để thu hồi tự động các đối tượng khi không còn cần thiết
  - GC không nhất thiết hoạt động với mọi đối tượng (không nhất thiết phải giải phóng bộ nhớ)
  - Không đảm bảo việc phương thức hủy luôn hoạt động
- GC tăng tốc độ phát triển và tăng tính ổn định của ứng dụng
  - Không phải viết mã giải phóng đối tượng
  - Do đó, không bao giờ "quên giải phóng đối tượng"

GC hoạt động như thế nào?

- Sử dụng cơ chế đếm
  - mỗi đối tượng có một số đếm các tham chiếu trỏ tới
  - giải phóng đối tượng khi số đếm = 0

```
MyDate openDate = new MyDate(2,9,2019);
MyDate startDate = new MyDate(10,10,2020);
openDate = startDate;
```

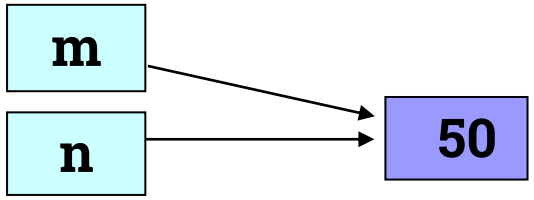


GC (tiếp)

- Java cung cấp phương thức hủy finalize()
  - Không nhất thiết được thực hiện do hệ thống không phải luôn luôn thực hiện việc giải phóng bộ nhớ
  - Ứng dụng finalize() trong debug chương trình: kiểm tra trạng thái cuối cùng của đối tượng
- Có thể yêu cầu hệ thống giải phóng bộ nhớ bằng phương thức System.gc()

Truyền tham số và nhận giá trị trả về

- Truyền giá trị
  - Giá trị tham số được đưa vào stack
  - Là cơ chế duy nhất được dùng trong Java
    - Java không hỗ trợ truyền theo tham chiếu
- Hai loại tham số
  - Dữ liệu nguyên thủy
    - giá trị tham số được sao chép
    - các tham số có thể là hằng, ví dụ, 10, “abc”…
  - Tham chiếu đối tượng
    - tham chiếu (nơi lưu đối tượng) được sao chép
    - KHÔNG sao chép nội dung đối tượng



Truyền tham số và nhận giá trị trả về

- Cách thức hoạt động
  - Tham số tương ứng với một biến địa phương để lưu giá trị truyền vào

Định nghĩa hàm

```
ReturnType foo(DataType v) {
   //processing v
}
```

Gọi hàm

```
...
foo(u); // u có kiểu DataType
```

tương đương

v là biến địa phương

điều gì xảy ra khi DataType là
- kiểu nguyên thủy?
- kiểu tham chiếu?

```
{
  DataType v = u;
  //processing v
}
```

Truyền tham số và nhận giá trị trả về

```
class Date {
    int year, month, day;
    public Date(int y, int m, int d) {
        year = y; month = m; day = d;
    }
    public void copy(Date d) {
        d.year = year;
        d.month = month;
        d.day = day;
    }
    public Date copy() {
        return new Date(day, month, year);
    }
...
}
```

y, m, d có kiểu nguyên thủy và sẽ nhận giá trị truyền vào

d là tham chiếu, sẽ nhận giá trị truyền vào là nơi lưu trữ đối tượng (object location)

trả về tham chiếu đến đối tượng vừa được tạo. Lưu ý, giá trị này là nơi lưu đối tượng, không phải nội dung đối tượng

Truyền tham số và nhận giá trị trả về

```
...
int thisYear = 2010;
Date d1 = new Date(thisYear, 9, 26);
```

```
class Date {
    int year, month, day;
    public Date(int y, int m, int d) {
        year = y; month = m; day = d;
    }
    public void copy(Date d) {
        d.year = year;
        d.month = month;
        d.day = day;
    }
    public Date copy() {
        return new Date(day, month, year);
    }
...
}
```

```
y = thisYear;
m = 9;
d = 26;
year = y;
month = m;
day = d;
```

Truyền tham số và nhận giá trị trả về

```
...
Date d1 = new Date(thisYear, 9, 26);
Date d2 = new Date(2000, 1, 1);
d1.copy(d2);
```

```
class Date {
    int year, month, day;
    public Date(int y, int m, int d) {
        year = y; month = m; day = d;
    }
    public void copy(Date d) {
        d.year = year;
        d.month = month;
        d.day = day;
    }
    public Date copy() {
        return new Date(day, month, year);
    }
...
}
```

```
d = d2;
d.year = year;
d.month = month;
d.day = day;
```

Truyền tham số và nhận giá trị trả về

```
...
Date d2 = new Date(2000, 1, 1);
Date d3 = d2.copy();
```

```
class Date {
    int year, month, day;
    public Date(int y, int m, int d) {
        year = y; month = m; day = d;
    }
    public void copy(Date d) {
        d.year = year;
        d.month = month;
        d.day = day;
    }
    public Date copy() {
        return new Date(day, month, year);
    }
...
}
```

```
Date temp =
  new Date(d2.day, d2.month, d2.year);
d3 = temp;
```

Tham chiếu “this”

- Tham chiếu this trỏ đến chính đối tượng đó
- Công dụng của this
  - tham chiếu tường minh đến thuộc tính và phương thức của đối tượng
  - truyền tham số và trả lại giá trị
  - dùng để gọi constructor

"this" để tham chiếu tường minh

```
class Date {
    int year, month, day;
    public Date(int year, int month, int day) {
        this.year = year;
        this.month = month;
        this.day = day;
    }
    public void copy(Date d) {
        d.year = year;
        d.month = month;
        d.day = day;
    }
...
}
```

“this” làm tham số

```
class Document {
    Viewer vi; //reference to the document’s viewer
    ...
    Document(Viewer v) {
        vi = v;
        ...
    }
    void display() {
        //ask the object’s viewer
        //…to display the current document
        vi.display(this);
    }
...
}
```

"this" là giá trị trả về

```
class Counter {
    private int c = 0;
    public Counter increase() {
        c++;
        return this;
    }
    public int getValue() {
        return c;
    }
}
...
Counter count = new Counter();
System.out.println(count.increase().increase().getValue());
```

"this" để gọi phương thức tạo

- Phương thức tạo chỉ được gọi từ trong phương thức tạo khác và chỉ được gọi một lần (ở lệnh đầu tiên)

```
class Date {
    private int year, month, day;

    public Date(int y, int m, int d) { ... }

    // copy constructor
    Date(Date d) {
        this(d.year, d.month, d.day);
        System.out.println("copy constructor called");
    }
...
}
```

Phương thức và thuộc tính tĩnh

- Phương thức và thuộc tính có thể được khai báo là thành phần tĩnh (static)
  - Độc lập với đối tượng
  - Có thể được truy cập không qua đối tượng
    - Sử dụng tên lớp
- Thuộc tính tĩnh
  - Thuộc về lớp
  - Được chia sẽ giữa các đối tượng của lớp

```java
public class Dummy {
   // number of Dummy objects
   static int counter = 0;
   static int count() {
      return counter;
   }

   private String name;

   public Dummy(String name) {
      counter++;
      this.name = name;
   }

   //main function to test Dummy class
   public static void main(String args[]) {
      System.out.println(Dummy.count());
      Dummy d1 = new Dummy ("First Dummy");
      System.out.println(d1.count());
      Dummy d2 = new Dummy ("Second Dummy");
      System.out.println(d1.count());
   }
}
```

Dummy.counter được chia sẻ và được cập nhật bởi các đối tượng Dummy.

```
0
1
2
```

Dummy.count() có thể được gọi qua lớp, không cần đối tượng.

Phương thức và thuộc tính tĩnh

- Phương thức tĩnh
  - Không thể truy cập các thành phần thông thường khác (non-static)
  - Không thể gọi các phương thức non-static
- Tại sao?

```
class Hello{
            public void sayHello(String msg){
                  System.out.println(msg);
            }
}
```

```
public class HelloTestDrive{
            public static void main(String[] args){
                  Hello h=new Hello();//tạo đối tượng
                  h.sayHello("Hello, world");
            }
}
```

```
class Hello{
            public static void sayHi(String msg)
{
                  System.out.println(msg);
            }
            public void sayHello(String msg){
                  System.out.println(msg);
            }
}
```

```
public class HelloTestDrive{
            public static void main(String[] args){
                  Hello.sayHi("Hi, there!");

                  Hello h=new Hello();
                  h.sayHello("Hello");
                  h.sayHi("Hello");
            }
}
```

```
class Hello{
            public static String message="Hello, world";
            public static void sayHi(msg) {
                  System.out.println(msg);
            }
      }
```

```
public class HelloTestDrive{
            public static void main(String[] args){
                  Hello.sayHi("Hi!");

                  System.out.println(Hello.message);

            }
}
```

Gói các lớp đối tượng (package)

- Các lớp đối tượng được chia thành các gói
  - nếu không khai báo thì các lớp thuộc gói default
  - các lớp trong cùng một tệp mã nguồn luôn thuộc cùng một gói
- Tồn tại mức truy cập gói (package)
  - mức package là mặc định (nếu không khai báo tường minh là public hay private)
  - các đối tượng của các lớp thuộc cùng gói có thể truy cập đến thành phần non-private của nhau
  - chỉ có thể tạo (new) đối tượng của lớp được khai báo là public của gói khác

Sử dụng Gói như thế nào?

```
//Hello.java:

class HelloMsg {
    void sayHello() {
        System.out.println(”Hello, world!”);
    }
}

public class Hello {
    public static void main(String[] args) {
        HelloMsg msg = new HelloMsg();
        msg.sayHello();
    }
}
```

Làm thế nào để đưa HelloMsg vào một gói?

Khai báo Gói

- Lệnh package xuất hiện ngay ở dòng đầu tiên trong tệp

khai báo gói bằng lệnh package với tên gói. Phần còn lại của tệp sẽ thuộc về cùng một gói.

```
// HelloMsg.java
package dse;

public class HelloMsg {
   public void sayHello() {
      System.out.println("Hello, world!");
   }
}
```

Được khai báo public nên có thể được sử dụng ngoài phạm vi gói dse.

Hai cách sử dụng Gói

```java
//Hello.java
import dse.HelloMsg;

public class Hello {
    public static void main(String[] args) {
        HelloMsg msg = new HelloMsg();
        msg.sayHello();
    }
}
```

1. Sử dụng lệnh import để cung cấp các tên (names) trong gói và chỉ cần một lần.

2. Chỉ rõ gói cho mỗi lần gọi.

```java
//Hello.java
public class Hello {
    public static void main(String[] args) {
        dse.HelloMsg msg = new dse.HelloMsg();
        msg.sayHello();
    }
}
```

dse/hello/Hello.java:

```java
package dse.hello;
public class Hello {
    public void sayHello() {
        System.out.println("Hello, world!");
    }
}
```

dse/test/HelloTestDrive.java:

```
import dse.hello.Hello;

public class Hello {
    public static void main(String[] args) {
        Hello h = new Hello();
        h.sayHello();
    }
}
```

Biên dịch và thực thi

- Biên dịch

  javac HelloMsg.java –d

  javac Hello.java

  javac dse/test/HelloTestDrive.java

- Thực thi

  java Hello

  java dse.test.HelloTestDrive

Hợp thành (composition)

- Các đối tượng có thể chứa các đối tượng khác ở dạng thuộc tính
- Các thuộc tính kiểu tham chiếu này phải được khởi tạo sử dụng toán tử new hoặc phép gán

Hợp thành

Dữ liệu có kiểu không nguyên thủy

```
public class Person {
   private String name;
   private Date birthDate;

   public Person(String name, Date birthDate) {
      this.name = name;
      this.birthDate = birthDate;
   }

   public String toString() {
      return String.format("%s: Birthday: %s",name,birthDate);
   }
}
```

```
...
Date d = new Date (31, 12, 1999);
Person p1 = new Person ("Bob", d);

Person p2 = new Person ("Alice", new Date (1, 1, 2000));
```

Đối tượng được chứa phải được tạo.

Get/Set thuộc tính tham chiếu

```
class Person {
…
    public MyDate getBirthday() {
        return birthday;
    }
}

Person p = new Person(...);
MyDate d = p.getBirthday();
d.setYear(1900);
```

Get/Set bằng khởi tạo sao chép

```
class Person {
    private String name;
    private MyDate birthday;
    public Person(String s, MyDate d) {
        name = s;
        birthday = new MyDate(d);
    }
    public MyDate getBirthday() {
        return new MyDate(birthday);
    }
    public void setBirthday(MyDate d) {
        birthday = new MyDate(d);
    }
...
}
```

Cách nào đúng???

Vào ra từ luồng dữ liệu chuẩn

- Ba đối tượng luồng được tạo tự động khi thực thi chương trình
  - **System.out:** đối tượng luồng ra chuẩn
    - thường cho phép chương trình xuất dữ liệu ra màn hình
  - **System.err:** đối tượng luồng lỗi chuẩn
    - thường cho phép chương trình xuất thông báo lỗi ra màn hình
  - **System.in:** đối tượng luồng vào chuẩn
    - thường cho phép chương trình nhận dữ liệu (bytes) từ bàn phím
- Có thể được điều hướng để được gửi hoặc đọc từ các nguồn khác, chẳng hạn từ tệp trên đĩa
  - Sử dụng phương thức setIn(), setOut(), setErr()
  - Tại cửa sổ lệnh (chỉ input và output):

    C:\> input.dat > java AJavaProgram > output.dat

Nhập dữ liệu từ luồng vào chuẩn

- InputStream: lớp đối tượng ứng với luồng vào chuẩn
  - System.in: đối tượng tương ứng
  - chưa có phương thức nhập dữ liệu
- Scanner: nhập dữ liệu kiểu nguyên thủy và xâu ký tự
  - next(): nhập xâu ký tự
  - nextType(): nhập dữ liệu kiểu Type
  - hasNext(), hasNextType(): kiểm tra xem còn dữ liệu không

Ví dụ: Nhập liệu từ luồng vào chuẩn

```
// import the wrapper class
import java.util.Scanner;
...
// create Scanner to get input from keyboard
Scanner sc = new Scanner( System.in );

// read a word
System.out.println(sc.next());

// read an integer
int i = sc.nextInt();

// read a series of big intergers
while (sc.hasNextLong()) {
   long aLong = sc.nextLong();
}
```

Tham số dòng lệnh

CmdLineParas.java:

```
public class CmdLineParas {
public static void main(String[] args) {
    for (int i=0; i<args.length; i++)
        System.out.println(args[i]);
    }
}
```

Ví dụ:

```
#java CmdLineParas hello world
hello
world
```